Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: mini ✓ Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests Create pull request

2 commits 2 files changed 1 contributor

Commits on Nov 3, 2026

ミニゲームと砲
c0a25091f5 committed 30 minutes ago b0b104b

ミニゲームと砲
c0a25091f5 committed 12 minutes ago 83ae7b7

Showing 2 changed files with 198 additions and 107 deletions. Split Unified

65 README.md

```
...  ...  @@ -1,20 +1,22 @@
1    1    # ProjExD_Group16
2         - #ooo 大戦争
     2    + # 工科大戦争
3    3
4    4    ## 実行環境の必要条件
5    5    * python >= 3.10
6    6    * pygame >= 2.1
7    7
8    8    ## ゲームの概要
9         - * （どんなゲームか）
10        - * 参考URL：[サイトタイトル](https://...)
     9    + * 右側の自陣からキャラクターを出撃させ、左側の敵陣を攻略するタワーディフェンスゲーム
     10   + * 参考：スマートフォンゲーム「にゃんこ大戦争」（PONOS）
11   11
12   12   ## ゲームの遊び方
13        - * （操作方法など）
     13   + * 数字キー1～3、または画面下のボタンのクリックでユニットを出撃させる
     14   + * お金は時間とともに増える。お金が足りない間やクールダウン中は出撃できない
     15   + * 敵陣のHPを0にすれば勝利、自陣のHPが0になれば敗北
14   16
15   17   ## ゲームの実装
```

```
16 18   ### 共通基本機能
17    - * 背景画像と主人公キャラクターの描画
   19 + * 出撃システム（お金とクールダウン）、ユニットの移動・戦闘、城のHP、勝利判定、UI表示
18 20
19 21   ### 分担追加機能
20 22   * 機能名（担当：名前）：説明
32 34
33 35   追加機能５（こざわ）：お金稼ぎミニゲーム
34 36   ➡画面上から落ちてくるコインをクリックでキャッチすると、お金が増えます。収入を待つだけでなく、操作で稼ぐ手応えが出ます。実装は Coin クラスのスプライトと衝突判定です。
   37 +
   38 + 仕様詳細
   39 + コインの生成
   40 +
   41 + 指定フレーム間隔（COIN_INTERVAL = 180：約3.6秒ごと）で1枚ずつ生成。
   42 +
   43 + 画面上部のランダムなX座標（50px ～ 画面幅 - 50px）から出現。
   44 +
   45 + 動作・挙動
   46 +
   47 + 画面下部（地面の高さ GROUND_Y）に向かって一定速度（1.5 ～ 3.0のランダム速度）で落下。
   48 +
   49 + 地面に到達すると自動的に消去（ヒットせず見逃した場合）。
   50 +
   51 + クリック判定（キャッチ）
   52 +
   53 + マウス左クリック時にコインの範囲内（collidepoint）であればコインを獲得。
   54 +
   55 + 1回のクリックでキャッチできるのは1枚のみ。
   56 +
   57 + コインをキャッチした場合、出撃ボタンなどの他のクリック処理はキャンセルされる。
   58 +
   59 + 報酬
   60 +
   61 + コイン1枚につき 50円 を所持金（wallet.money）に追加（上限 max_money を超えない範囲）。
   62 +
   63 + にゃんこ砲機能
   64 + 概要
   65 + 時間の経過でゲージがチャージされ、満タン時に自陣（城）の上に表示されるボタンを押すことで、画面上のすべての敵ユニットにまとめて大ダメージを与える必殺技要素。
   66 +
   67 + 仕様詳細
   68 + チャージ仕様
   69 +
   70 + max_charge（初期設定：1000フレーム）経過するとチャージ完了。
   71 +
   72 + チャージ中は自陣の城の上に青色の進捗ゲージを表示。
   73 +
   74 + UI表示（準備完了時）
   75 +
   76 + チャージが満タンになると、自陣の城の上に黄色の「準備OK!」ボタンが表示。
   77 +
   78 + 発射・効果
   79 +
   80 + 「準備OK!」ボタンをマウス左クリックすると発射。
   81 +
   82 + 発射時、画面上に存在する全ての敵ユニットに即座に 200ダメージ を与える。
   83 +
```

```
84 + 発射後、チャージ量は0に戻り、再チャージを開始。
85 +
86 + ビーム演出（エフェクト）nyan
87 +
88 + 発射から15フレーム間、自陣から画面左端へ向かって貫く紫色の極太ビーム（半透明描画）を表示。
89   ### メモ
90   * （変数名や設計のルールなど、メンバーへの連絡）
   -
```

240 nyanko base.py

```
...  ...  @@ -1,11 +1,13 @@
1    1    """
2         - にゃんこ大戦争風ゲームの共通基本機能（ベース部分）
     2    + にゃんこ大戦争風ゲーム（ベース＋コインキャッチ＋にゃんこ砲）
3    3
4    4    右側の自陣から味方ユニット（ネコ）を出撃させ，左側の敵陣を攻略するゲーム．
5    5    お金は時間とともに増え，お金とクールダウンの許す範囲でユニットを出撃できる．
6    6    ユニットは敵に当たると立ち止まって攻撃し，敵陣のHPを0にすれば勝利．
7    7
8    8    操作：数字キー1～3，または画面下のボタンのクリックでユニットを出撃
     9    +    空から落ちてくるコインをクリックでキャッチするとお金が増加
     10   +    自陣上の「準備OK！」をクリックするとにゃんこ砲発射！
9    11   """
10   12   import os
11   13   import random
19   21   GROUND_Y = 500  # 地面の高さ（ユニットの足元のy座標）
20   22   FPS = 50
21   23   ENEMY_INTERVAL = 150  # 敵が出現する間隔（フレーム数）
     24   + COIN_INTERVAL = 180   # コインが出現する間隔（フレーム数）
22   25   BASE_HP = 1000
23   26   os.chdir(os.path.dirname(os.path.abspath(__file__)))
24   27
25        - # 味方ユニットの性能（追加機能として種類を増やしやすいように辞書で管理）
     28   + # 味方ユニットの性能
26   29   CAT_SPECS = [
27   30       {"name": "ネコ", "cost": 50, "hp": 100, "atk": 10, "speed": 2.0,
28   31        "range": 20, "interval": 40, "cooldown": 100,
47   50
48   51
49   52   def get_font(size: int) -> pg.font.Font:
50        -     """
51        -     日本語を表示できるフォントを探して返す
52        -     引数 size：フォントサイズ
53        -     戻り値：Fontオブジェクト（見つからない場合は標準フォント）
54        -     """
55   53       name = pg.font.match_font(
56   54           "meiryo,yugothic,msgothic,notosanscjkjp,hiraginosans,takaopgothic"
57   55       )
58   56       return pg.font.Font(name, size)
59   57
60        -
61   58   def draw_hp_bar(screen: pg.Surface, rct: pg.Rect, hp: int, max_hp: int) -> None:
62        -     """
63        -     対象の頭上にHPバーを描画する
64        -     引数 screen：画面Surface
65        -     引数 rct：対象のRect
```

```
66        -     引数 hp：現在のHP
67        -     引数 max_hp：最大HP
68        -     """
69    59        ratio = max(0, hp) / max_hp
70    60        bar = pg.Rect(rct.left, rct.top - 10, rct.width, 6)
71    61        pg.draw.rect(screen, (80, 80, 80), bar)
      62    +   pg.draw.rect(screen, (0, 0, 0), bar, 1)
72    63        pg.draw.rect(screen, (0, 200, 0), (bar.left, bar.top, int(bar.width * ratio),
      bar.height))
73    64
74    65
75    66    class Base:
76        -     """
77        -     自陣・敵陣（城）に関するクラス
78        -     """
79    67        def __init__(self, is_cat: bool) -> None:
80        -         """
81        -         引数 is_cat：Trueなら自陣（右側），Falseなら敵陣（左側）
82        -         """
83    68            self.hp = BASE_HP
84    69            self.max_hp = BASE_HP
85    70            self.image = pg.Surface((80, 200))
90    75            self.rect.centerx = WIDTH - 80 if is_cat else 80
91    76
92    77        def draw(self, screen: pg.Surface) -> None:
93        -         """
94        -         城とHPバーを描画する
95        -         引数 screen：画面Surface
96        -         """
97    78            screen.blit(self.image, self.rect)
98    79            draw_hp_bar(screen, self.rect, self.hp, self.max_hp)
99    80
100   81
101   82    class Unit(pg.sprite.Sprite):
102       -     """
103       -     味方・敵のユニットに関するクラス
104       -     """
105   83        def __init__(self, spec: dict, is_cat: bool, x: int) -> None:
106       -         """
107       -         引数 spec：ユニットの性能を表す辞書
108       -         引数 is_cat：Trueなら味方（左へ進む），Falseなら敵（右へ進む）
109       -         引数 x：出現位置のx座標（味方は右端，敵は左端をこの座標に合わせる）
110       -         """
111   84            super().__init__()
112   85            self.spec = spec
113   86            self.hp = spec["hp"]
114   87            self.dir = -1 if is_cat else +1
115       -         self.atk_timer = spec["interval"]  # 最初の1発はすぐ撃てるようにする
      88    +     self.atk_timer = spec["interval"]
116   89            self.image = pg.Surface(spec["size"])
117   90            self.image.fill(spec["color"])
118   91            pg.draw.rect(self.image, (0, 0, 0), self.image.get_rect(), 2)
122   95                self.rect.right = x
123   96            else:
124   97                self.rect.left = x
125       -         self.x = float(self.rect.centerx)  # 小数の速度を扱うため，x座標は別に保持する
      98    +     self.x = float(self.rect.centerx)
126   99
```

```
127 100         def can_reach(self, rct: pg.Rect) -> bool:
128             -       """
129             -       対象のRectが攻撃射程内にあるかを判定する
130             -       引数 rct：敵ユニットまたは敵陣のRect
131             -       戻り値：射程内ならTrue
132             -       """
133 101             if self.dir < 0:
134 102                 return rct.right >= self.rect.left - self.spec["range"]
135 103             return rct.left <= self.rect.right + self.spec["range"]
136 104
137 105         def find_target(self, opponents: pg.sprite.Group, base: Base) -> "Unit | Base | None":
138             -       """
139             -       攻撃対象を探す（射程内のユニットのうち最も近いもの，いなければ敵城）
140             -       引数 opponents：相手側ユニットのグループ
141             -       引数 base：相手側の城
142             -       戻り値：攻撃対象（いなければNone）
143             -       """
144 106             cands = [o for o in opponents if self.can_reach(o.rect)]
145 107             if cands:
146 108                 return min(cands, key=lambda o: abs(o.rect.centerx - self.rect.centerx))
149 111             return None
150 112
151 113         def update(self, opponents: pg.sprite.Group, base: Base) -> None:
152             -       """
153             -       HP判定，攻撃，移動を行う
154             -       引数 opponents：相手側ユニットのグループ
155             -       引数 base：相手側の城
156             -       """
157 114             if self.hp <= 0:
158 115                 self.kill()
159 116                 return
168 125                 self.atk_timer = 0
169 126
170 127
171             - class Wallet:
    128         + class Coin(pg.sprite.Sprite):
    129         +     def __init__(self, reward: int = 50) -> None:
    130         +         super().__init__()
    131         +         self.reward = reward
    132         +         self.radius = 16
    133         +         self.image = pg.Surface((self.radius * 2, self.radius * 2), pg.SRCALPHA)
    134         +         pg.draw.circle(self.image, (255, 215, 0), (self.radius, self.radius), self.radius)
    135         +         pg.draw.circle(self.image, (218, 165, 32), (self.radius, self.radius), self.radius, 2)
    136         +
    137         +         self.rect = self.image.get_rect()
    138         +         self.rect.centerx = random.randint(50, WIDTH - 50)
    139         +         self.rect.bottom = 0
    140         +         self.speed = random.uniform(1.5, 3.0)
    141         +
    142         +     def update(self) -> None:
    143         +         self.rect.y += self.speed
    144         +         if self.rect.top > GROUND_Y:
    145         +             self.kill()
    146         +
    147         +
```

```
148 + class CatCannon:
172 149       """
173         -     お金（所持金）に関するクラス
    150 +     にゃんこ砲クラス
174 151       """
    152 +     def __init__(self, cat_base: Base) -> None:
    153 +         self.cat_base = cat_base
    154 +         self.max_charge = 1000  # チャージ完了までのフレーム数（10秒）
    155 +         self.charge = 0         # 現在のチャージ量
    156 +         self.damage = 200       # 敵全体へのダメージ
    157 +         self.beam_timer = 0     # ビームの描画用タイマー
    158 +
    159 +         # 自陣の城の上に表示する「準備OK！」UIのRect
    160 +         self.btn_rect = pg.Rect(0, 0, 110, 40)
    161 +         self.btn_rect.centerx = cat_base.rect.centerx
    162 +         self.btn_rect.bottom = cat_base.rect.top - 15
    163 +
    164 +     def is_ready(self) -> bool:
    165 +         """チャージが満タンかどうか"""
    166 +         return self.charge >= self.max_charge
    167 +
    168 +     def update(self) -> None:
    169 +         """チャージ進行およびビームタイマー更新"""
    170 +         if not self.is_ready():
    171 +             self.charge += 1
    172 +         if self.beam_timer > 0:
    173 +             self.beam_timer -= 1
    174 +
    175 +     def try_fire(self, pos: tuple[int, int], enemies: pg.sprite.Group) -> bool:
    176 +         """
    177 +         「準備OK！」ボタンが押されたらビームを発射し、画面内の敵全体にダメージ
    178 +         """
    179 +         if self.is_ready() and self.btn_rect.collidepoint(pos):
    180 +             self.charge = 0
    181 +             self.beam_timer = 15  # ビーム表示フレーム数
    182 +             # 全敵ユニットへダメージを与える
    183 +             for enemy in enemies:
    184 +                 enemy.hp -= self.damage
    185 +             return True
    186 +         return False
    187 +
    188 +     def draw(self, screen: pg.Surface, font: pg.font.Font) -> None:
    189 +         """UI（準備OKボタン / ゲージ）とビームの描画"""
    190 +         # --- ビームエフェクト描画 ---
    191 +         if self.beam_timer > 0:
    192 +             beam_rect = pg.Rect(0, GROUND_Y - 80, self.cat_base.rect.centerx, 80)
    193 +             beam_surface = pg.Surface((beam_rect.width, beam_rect.height),
        pg.SRCALPHA)
    194 +             # (R, G, B, 透明度A)
    195 +             beam_surface.fill((200, 100, 255, 180))  # 紫色の外光
    196 +             pg.draw.rect(beam_surface, (230, 180, 255, 230), (0, 15, beam_rect.width,
        50))  # 明るい紫色の芯
    197 +             screen.blit(beam_surface, beam_rect)
    198 +
    199 +         # --- UI描画（城の上） ---
    200 +         if self.is_ready():
    201 +             # 準備OKボタン（点滅っぽく黄色で強調）
    202 +             pg.draw.rect(screen, (255, 215, 0), self.btn_rect, border_radius=8)
```

```
+            pg.draw.rect(screen, (255, 255, 255), self.btn_rect, 3, border_radius=8)
+            text = font.render("準備OK!", True, (0, 0, 0))
+            screen.blit(text, text.get_rect(center=self.btn_rect.center))
+        else:
+            # チャージ中ゲージ
+            gauge_bg = pg.Rect(self.btn_rect.left, self.btn_rect.bottom - 12, self.btn_rect.width, 12)
+            ratio = self.charge / self.max_charge
+            gauge_fg = pg.Rect(gauge_bg.left, gauge_bg.top, int(gauge_bg.width * ratio), gauge_bg.height)
+
+            pg.draw.rect(screen, (60, 60, 60), gauge_bg, border_radius=4)
+            pg.draw.rect(screen, (100, 200, 255), gauge_fg, border_radius=4)
+            pg.draw.rect(screen, (0, 0, 0), gauge_bg, 1, border_radius=4)
+
+
+class Wallet:
     def __init__(self) -> None:
         self.money = 100.0
         self.max_money = 1000
-        self.income = 0.5  # 1フレームあたりの増加量
+        self.income = 0.5

     def update(self) -> None:
-        """
-        お金を時間経過で増やす（上限あり）
-        """
         self.money = min(self.max_money, self.money + self.income)

+    def add_money(self, amount: int) -> None:
+        self.money = min(self.max_money, self.money + amount)
+
     def pay(self, cost: int) -> bool:
-        """
-        所持金が足りれば支払う
-        引数 cost：支払う金額
-        戻り値：支払えたらTrue，足りなければFalse
-        """
         if self.money < cost:
             return False
         self.money -= cost
         return True


 def make_buttons() -> list[pg.Rect]:
-    """
-    出撃ボタンのRectのリストを作る
-    戻り値：味方ユニットの種類ごとのボタンRect
-    """
     return [pg.Rect(20 + i * 170, 540, 160, 90) for i in range(len(CAT_SPECS))]

-
 def try_spawn(idx: int, wallet: Wallet, cooldowns: list[int],
               cats: pg.sprite.Group, cat_base: Base) -> None:
-    """
-    お金とクールダウンの条件を満たせば，味方ユニットを出撃させる
-    引数 idx：CAT_SPECSの番号
-    引数 wallet：所持金
```

```
-     引数 cooldowns：ユニットごとの残りクールダウン（フレーム数）
-     引数 cats：味方ユニットのグループ
-     引数 cat_base：自陣
-     """
      spec = CAT_SPECS[idx]
      if cooldowns[idx] > 0 or not wallet.pay(spec["cost"]):
          return
      cats.add(Unit(spec, True, cat_base.rect.left))
      cooldowns[idx] = spec["cooldown"]

-
  def draw_ui(screen: pg.Surface, font: pg.font.Font, wallet: Wallet,
              cooldowns: list[int], buttons: list[pg.Rect]) -> None:
-     """
-     所持金表示と出撃ボタンを描画する
-     引数 screen：画面Surface
-     引数 font：表示用フォント
-     引数 wallet：所持金
-     引数 cooldowns：ユニットごとの残りクールダウン
-     引数 buttons：出撃ボタンのRectリスト
-     """
      for i, (spec, btn) in enumerate(zip(CAT_SPECS, buttons)):
          usable = cooldowns[i] == 0 and wallet.money >= spec["cost"]
          pg.draw.rect(screen, (255, 230, 120) if usable else (150, 150, 150), btn)
-         if cooldowns[i] > 0:  # クールダウン中は残り時間を暗いゲージで示す
+         if cooldowns[i] > 0:
              h = int(btn.height * cooldowns[i] / spec["cooldown"])
              pg.draw.rect(screen, (90, 90, 90), (btn.left, btn.top, btn.width, h))
          pg.draw.rect(screen, (0, 0, 0), btn, 3)
      money = font.render(f"お金 {int(wallet.money)} / {wallet.max_money}円", True, (0,
  0, 0))
      screen.blit(money, (WIDTH - money.get_width() - 20, 20))

-
  def draw_field(screen: pg.Surface) -> None:
-     """
-     背景（空と地面）を描画する
-     引数 screen：画面Surface
-     """
      screen.fill((135, 206, 235))
      pg.draw.rect(screen, (150, 110, 70), (0, GROUND_Y, WIDTH, HEIGHT - GROUND_Y))
      pg.draw.line(screen, (0, 0, 0), (0, GROUND_Y), (WIDTH, GROUND_Y), 3)

-
  def show_result(screen: pg.Surface, font: pg.font.Font, message: str) -> None:
-     """
-     勝敗の結果を3秒間表示する
-     引数 screen：画面Surface
-     引数 font：表示用フォント
-     引数 message：表示する文字列
-     """
      overlay = pg.Surface((WIDTH, HEIGHT))
      overlay.set_alpha(150)
      screen.blit(overlay, (0, 0))
      pg.display.update()
      time.sleep(3)

-
```

```
274 277   def main() -> None:
275 278       pg.display.set_caption("ネコ大戦争（仮）")
276 279       screen = pg.display.set_mode((WIDTH, HEIGHT))
277 280       font = get_font(26)
    281 +     btn_font = get_font(20)
278 282       big_font = get_font(90)
279 283       clock = pg.time.Clock()
280 284
281 285       cat_base, enemy_base = Base(True), Base(False)
282 286       cats, enemies = pg.sprite.Group(), pg.sprite.Group()
    287 +     coins = pg.sprite.Group()
    288 +     cannon = CatCannon(cat_base)  # にゃんこ砲生成
283 289       wallet = Wallet()
284 290       cooldowns = [0] * len(CAT_SPECS)
285 291       buttons = make_buttons()
291 297                   return
292 298               if event.type == pg.KEYDOWN and pg.K_1 <= event.key < pg.K_1 +
    len(CAT_SPECS):
293 299                   try_spawn(event.key - pg.K_1, wallet, cooldowns, cats, cat_base)
    300 +
294 301               if event.type == pg.MOUSEBUTTONDOWN and event.button == 1:
295     -                 for i, btn in enumerate(buttons):
296     -                     if btn.collidepoint(event.pos):
297     -                         try_spawn(i, wallet, cooldowns, cats, cat_base)
    302 +                 # 1. にゃんこ砲発射判定
    303 +                 if cannon.try_fire(event.pos, enemies):
    304 +                     continue
    305 +
    306 +                 # 2. コインキャッチ判定
    307 +                 clicked_coin = False
    308 +                 for coin in list(coins):
    309 +                     if coin.rect.collidepoint(event.pos):
    310 +                         wallet.add_money(coin.reward)
    311 +                         coin.kill()
    312 +                         clicked_coin = True
    313 +                         break
    314 +
    315 +                 # 3. 出撃ボタン判定
    316 +                 if not clicked_coin:
    317 +                     for i, btn in enumerate(buttons):
    318 +                         if btn.collidepoint(event.pos):
    319 +                             try_spawn(i, wallet, cooldowns, cats, cat_base)
298 320
299 321           wallet.update()
    322 +         cannon.update()  # にゃんこ砲のチャージ更新
    323 +
300 324           for i in range(len(cooldowns)):
301 325               cooldowns[i] = max(0, cooldowns[i] - 1)
    326 +
    327 +         # 敵の定期出現
302 328           if tmr % ENEMY_INTERVAL == ENEMY_INTERVAL - 1:
303 329               spec = random.choices(ENEMY_SPECS, weights=[3, 1])[0]
304 330               enemies.add(Unit(spec, False, enemy_base.rect.right))
    331 +
    332 +         # コインの定期出現
    333 +         if tmr % COIN_INTERVAL == COIN_INTERVAL - 1:
    334 +             coins.add(Coin(reward=50))
    335 +
```

```
305 336         cats.update(enemies, enemy_base)
306 337         enemies.update(cats, cat_base)
    338 +       coins.update()
307 339
    340 +       # 描画処理
308 341         draw_field(screen)
309 342         enemy_base.draw(screen)
310 343         cat_base.draw(screen)
311 344         cats.draw(screen)
312 345         enemies.draw(screen)
    346 +       coins.draw(screen)
    347 +
313 348         for unit in list(cats) + list(enemies):
314 349             draw_hp_bar(screen, unit.rect, unit.hp, unit.spec["hp"])
    350 +
315 351         draw_ui(screen, font, wallet, cooldowns, buttons)
    352 +       cannon.draw(screen, btn_font)  # にゃんこ砲のUIとビームを描画
316 353
317 354         if enemy_base.hp <= 0:
318 355             show_result(screen, big_font, "勝利！")
319 356             return
320 357         if cat_base.hp <= 0:
321 358             show_result(screen, big_font, "敗北…")
322 359             return
    360 +
323 361         pg.display.update()
324 362         tmr += 1
325 363         clock.tick(FPS)
```